카테고리별 유사 스타일 추천 시스템

이진하

요약

본 보고서는 온라인 쇼핑 환경에서 텍스트 정보만으로 파악하기 어려운 사용자의 취향에 맞춘 의류의 시각적 스타일과 분위기를 반영하기 위해 CNN 기반 이미지 임베딩과 카테고리 필터링을 결합한 유사 스타일 추천 시스템을 구현하였다.
필루미네이트의 공식 온라인 쇼핑 사이트와 무신사의 필루미네이트 카테고리의 상품 데이터를 활용하여 상의, 하의, 겉옷, 모자, 액세서리의 5개 카테고리로 데이터셋을 재구성 하였으며, 실험 결과 학습 횟수(Epoch)를 20회로 설정했을 때 가장 안정적인 분류 성능과 높은 예측 확신도를 보였다. 본 시스템은 텍스트 의존도를 낮추고 이미지 자체의 시각정 특징(색감, 소재, 질감, 스타일 등)을 활용함으로써 사용자에게 직관적이고 유사도가 높은 상품을 효과적으로 추천할 수 있음을 확인하였다.
향후에는 단순한 추천을 넘어, 사용자가 직접 찍은 사진이나 웹상의 이미지를 통해 즉각적으로 상품을 찾아주는 구글의 AI 이미지 검색과 같은 지능형 쇼핑 서비스로의 발전 가능성을 확인하였다. 이미지 임베딩 기술 고도화를 통해 텍스트 검색의 한계를 극복한 차세대 커머스 환경을 구축할 수 있을 것으로 기대된다.

1.서론

1.1 연구배경

의류 브랜드인 필루미네이트를 기준으로 카테고리 기반 의상 추천 시스템을 구현하였다. 온라인 쇼핑 환경에서는 상품의 종류가 방대해짐에 따라 사용자가 원하는 스타일의 상품을 빠르게 탐색하기 어렵다는 문제가 있다. 본 프로젝트에서는 이러한 문제를 해결하기 위해 다음과 같은 세 가지 핵심 문제점 해결에 주목하였다.
첫 번째, 원하는 스타일의 상품을 빠르게 탐색하기 어렵다. 수많은 상품 중 사용자가 특정 스타일이나 디자인을 가진 제품을 일일이 찾아내는 데에는 많은 시간과 노력이 소모된다.
두 번째, 텍스트 정보만으로 상품의 정확한 분위기를 파악하기에 한계가 있다. 추상적인 텍스트 키워드로는 의류가 가진 색감, 소재, 질감, 스타일 차이를 완벽하게 전달할 수 없다.
세 번째, 부적절한 탐색 환경은 반복되는 비즈니스 손실로 이어진다. 사용자가 원하는 상품을 찾지 못해 이탈하는 과정이 반복되면 고객 만족도가 하락하고, 이는 잠재적 구매 기회의 상실이라는 경영적 손실로 이어질 수 있다.
따라서 본 보고서에서는 이러한 한계를 극복하고, 이미지 자체의 시각적 특징을 활용하여 사용자가 원하는 스타일을 연결해주는 시스템을 구축하고자 한다.

2. 시스템 설계

본 시스템은 실질적인 추천 성능을 확보하기 위해 데이터 수집 및 정제, CNN 기반 특징 추출, 카테고리 필터링 기반 추천의 3단계로 구성하였다.

먼저, 필루미네이트의 사이트에서 제공하는 기존 상품 카테고리 정보를 활용하여 상품 이미지를 상의(top) , 하의(bottom) , 겉옷(outerwear) , 모자(hat) , 악세서리(accessory) 다섯 개 카테고리로 분류하였다.

```
카테고리 작업 중: bottom -> data\bottom
Reading: https://filluminate.com/category/bottom/893/?page=1
Reading: https://filluminate.com/category/bottom/893/?page=2
Reading: https://filluminate.com/category/bottom/893/?page=3
Reading: https://filluminate.com/category/jeans/986/?page=1
Reading: https://filluminate.com/category/jeans/986/?page=2
Reading: https://filluminate.com/category/jeans/986/?page=3
Reading: https://filluminate.com/category/pants/987/?page=1
Reading: https://filluminate.com/category/pants/987/?page=2
Reading: https://filluminate.com/category/pants/987/?page=3
Reading: https://filluminate.com/category/shorts/988/?page=1
Reading: https://filluminate.com/category/shorts/988/?page=2
Reading: https://filluminate.com/category/shorts/988/?page=3
```

그림 2.1 카테고리별 데이터 수집

<그림 2.1>은 필루미네이트의 웹 구조를 분석하여 상의, 하의, 겉옷, 모자, 액세서리의 5개 핵심 카테고리를 설정하여 파이썬 기반 자동화 크롤러를 통해 각 카테고리에 해당하는 이미지를 폴더 단위로 수집하였다. 충분한 데이터를 확보하기 위해 페이지네이션을 탐색하도록 조정하여 다량의 상품 데이터를 자동으로 확보할 수 있도록 구현하였고, 서버 부하 방지를 위해 다운로드 간 지연시간을 설정하여 데이터 유실 없이 안정적으로 수집하였다.

수동 수집 방법에서 발생할 수 있는 데이터 누락을 방지하고, 단시간에 방대한 양의 이미지 데이터를 수집하여야 했기 때문에 많은 양의 데이터를 수집할 수 있는 자동화 프로그램으로 구축하였다.

표 2.1 카테고리별 파일 생성

카테고리	이미지 개수
accessory	81장
bottom	341장
hat	64장
outerwear	104장
top	261장
합계	851장

<표 2.1>은 크롤링 결과이다. 총 851장의 이미지는 딥러닝 모델의 초기 특징 추출을 학습하기에 충분한 규모이며, 카테고리별 분포를 통해 데이터셋이 구축되었다.

표 2.2 학습 / 테스트용 분리

카테고리	학습용	테스트용
accessory	64장	17장
bottom	272장	69장
hat	51장	13장
outerwear	83장	21장
top	208장	53장

모델의 객관적인 성능 검증을 위해 전체 데이터를 학습용(Train) 80%와 테스트용(Test) 20%의 비율로 분할하였다. 테스트 데이터는 모델 학습 과정에 관여하지 않는 데이터로 구성하여, 실제 서비스 환경에서의 추천 성능을 더 정확성있게 측정하고자 하였다.

3. 최적화 및 실험

모델 최적화 및 실험 과정으로는 모델의 학습 횟수(Epoch)를 5부터 시작하여 20까지 늘려가며 성능 변화를 단계적으로 확인하며 시스템을 개선하였다.

3.1 Epoch = 5

```
[1/5] Loss: 0.5597 | 성확도: 80.94%
[2/5] Loss: 0.1896 | 정확도: 92.98%
[3/5] Loss: 0.1084 | 정확도: 95.82%
[4/5] Loss: 0.0729 | 정확도: 97.16%
[5/5] Loss: 0.0411 | 정확도: 98.83%
```

그림 3.1 Epoch = 5

<그림 3.1>은 초기 실험으로 학습 횟수를 5로 하여 성능 변화를 보았다. Loss 값이 단계적으로 줄어드는 것을 확인 할 수 있었으며, 정확도도 80%부터 98%까지 늘어나는 것을 확인 할 수 있었다.

그림 3.2

<그림 3.2>에서 보시다시피, Epoch = 5 일 때 상의 이미지를 하의로 분류하거나, 아우터 이미지를 하의로 분류하는 등 카테고리 간 혼동이 빈번하게 발생하는 문제를 확인하였다. 이는 모델 학습에 충분히 정제된 형태가 아니었기 때문으로 판단하고, 이러한 문제를 해결하기 위해 이미지 데이터를 직접 확인하며 상의, 하의, 겉옷, 모자, 악세서리로 카테고리를 재구성하고 폴더 단위로 정리하였다.

3.2 Epoch = 10

```
[1/10] Loss: 0.4434 | 정확도: 84.45%
[2/10] Loss: 0.1760 | 정확도: 93.81%
[3/10] Loss: 0.1347 | 정확도: 94.82%
[4/10] Loss: 0.0826 | 정확도: 97.32%
[5/10] Loss: 0.0353 | 정확도: 98.33%
[6/10] Loss: 0.0326 | 정확도: 99.33%
[7/10] Loss: 0.0414 | 정확도: 98.49%
[8/10] Loss: 0.0137 | 정확도: 99.83%
[9/10] Loss: 0.0062 | 정확도: 100.00%
[10/10] Loss: 0.0223 | 정확도: 99.67%
```

그림 3.3 Epoch = 10

<그림 3.3>은 학습 횟수를 10으로 증가시켜 동일한 이미지 데이터를 학습시켰다. 학습이 진행됨에 따라 전반적으로 Loss값이 감소하고 정확도가 상승하는 추세를 보였다. 하지만, 마지막 10회차에서 Loss값이 소폭 상승하고 정확도가 100%에서 99.67%로 낮아지는 현상이 관찰되었다. 이러한 불안정성을 해결하기 위해 학습 횟수를 더 늘려보기로 하였다.

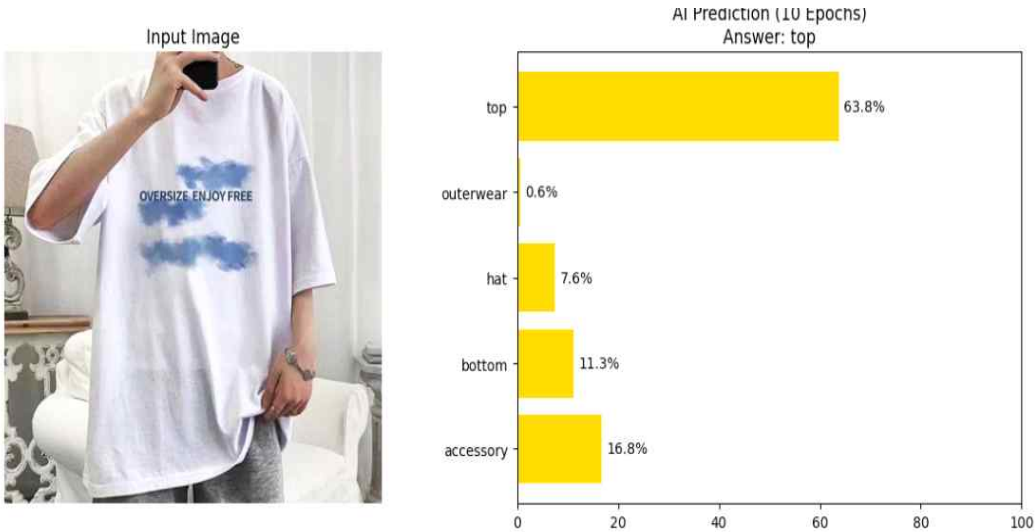

그림 3.4

<그림 3.4>는 학습 횟수를 늘리기 전, 성능 검사를 위해 실험해 보았다. 상의 이미지를 상의로 분류하며 결과 자체는 개선되었으나, 모델의 예측 확신도가 약 46.5%로 낮게 나타났다. 이는 모델이 카테고리를 구분할 수는 있으나, 아직 충분히 안정적으로 판단하지 못하는 단계로 해석하였다.

3.3 Epoch = 20

```
[1/20] Loss: 0.5756 | 정확도: 79.77%
[2/20] Loss: 0.3137 | 정확도: 87.96%
[3/20] Loss: 0.2216 | 정확도: 91.81%
[4/20] Loss: 0.1662 | 정확도: 94.48%
[5/20] Loss: 0.1110 | 정확도: 95.99%
[6/20] Loss: 0.0920 | 정확도: 96.15%
[7/20] Loss: 0.1195 | 정확도: 95.48%
[8/20] Loss: 0.0650 | 정확도: 97.66%
[9/20] Loss: 0.0536 | 정확도: 97.66%
[10/20] Loss: 0.0476 | 정확도: 98.49%
[11/20] Loss: 0.0415 | 정확도: 98.33%
[12/20] Loss: 0.0638 | 정확도: 97.83%
[13/20] Loss: 0.1233 | 정확도: 96.15%
[14/20] Loss: 0.1508 | 정확도: 94.31%
[15/20] Loss: 0.0834 | 정확도: 95.99%
[16/20] Loss: 0.0984 | 정확도: 95.82%
[17/20] Loss: 0.0957 | 정확도: 97.16%
[18/20] Loss: 0.0620 | 정확도: 98.49%
[19/20] Loss: 0.0658 | 정확도: 97.83%
[20/20] Loss: 0.0610 | 정확도: 98.16%
```

그림 3.5 Epoch = 20

<그림 3.5>에서 확인할 수 있듯이, 학습이 진행됨에 따라 Loss값이 안정적으로 낮은 수치에서 유지되며 정확도 또한 98%의 높은 수준을 지속적으로 기록하였다. 이전 실험(Epoch = 10)에서 나타났던 미세한 수치 진동이 완화되었다.

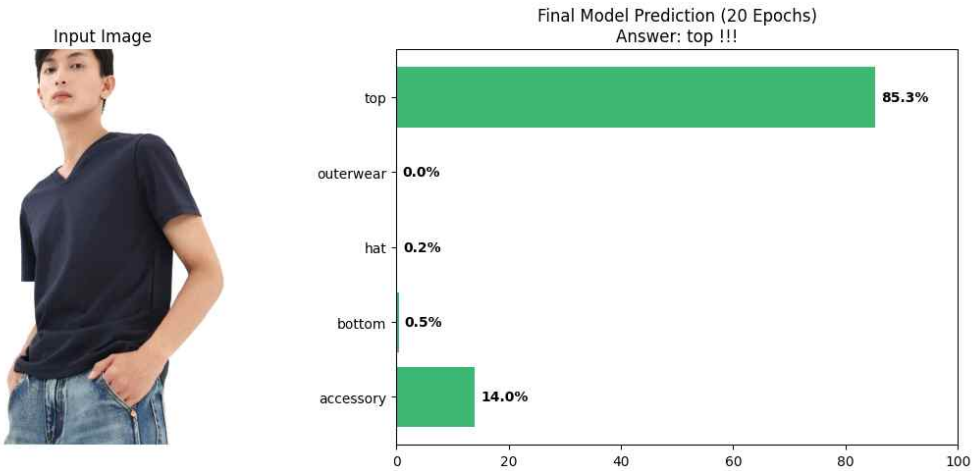

그림 3.6

Epoch = 20 에서는 기존에 사용한 이미지 뿐만 아니라 새 제품 이미지에 대해서도 카테고리를 안정적으로 분류하는 모습을 확인 할 수 있었다. 또한, 예측 확신도 역시 이전 단계보다 전반적으로 향상되어, 모델이 카테고리별 핵심 특징을 충분히

학습하였고, 성능 변화의 폭이 수렴하는 양상을 보임에 따라 본 프로젝트의 최종 추천 시스템 모델로 선정하였다. 이러한 실험을 통해 학습 횟수가 너무 적을 경우에는 충분한 특징 학습이 이루어지지 않으며, 반대로 일정 수준 이상에서는 성능 개선이 제한적임을 확인하였다. 이에 따라 성능과 안정성을 고려하여 Epoch = 20에서 학습을 종료하였다.

3.4 실험 결과

그림 3.7

그림 3.8

그림 3.9

그림 3.10

<그림 3.7>부터 <그림3.10> 까지 최종 모델을 적용하여 각 카테고리별 유사 아이템을 추천한 결과, 시각적 특징이 우수하게 반영됨을 확인하였다.
<그림 3.7>과 같이 입력 상품이 블루 셔츠일 경우, 유사한 색감과 실루엣을 가진 셔츠 아이템들이 우선적으로 추천되었으며, 유사도 순으로 순위를 매겨 추천된 것을 확인할 수 있었다.
<그림 3.8>에서는 비슷한 소재의 질감이나 색상의 옷이 추천되었지만, 후드티를 넣었는데 후드집업이 나오는 오류가 생겼다.
<그림 3.9>에서는 백팩 입력 시 유사한 크기와 형태를 가진 가방류가 추천되었지만, 선글라스가 포함하는 오류가 생겼다.
<그림 3.10>의 데님 팬츠에서는 워싱의 정도나 컬러감이 유사한 하의 아이템들이 효과적으로 추천된 것을 확인할 수 있었다.

4. 결론

본 보고서에서는 CNN 기반 이미지 임베딩과 카테고리 필터링을 결합한 의상 추천 시스템을 구현하였다. 텍스트 정보에 의존하지 않고, 이미지 자체의 시각적 특징을 활용함으로써 사용자가 선택한 상품과 유사한 스타일의 의류를 효과적으로 추천할 수 있음을 확인하였다. 이를 통해 이미지 기반 추천 시스템이 의류 쇼핑 환경에서 충분한 활용 가능성을 가진다는 점을 확인하였다.

5. 한계점 및 향후과제

이미지 유사도 기반 추천의 특성상, 촬영 배경이나 조명, 색감이 유사한 경우 세부 품목 간 혼재가 발생할 수 있다는 한계가 존재했다. 또한, 현재는 단일 이미지 입력을 기준으로 추천이 이루어지기 때문에, 사용자의 개인적인 취향이나 구매 이력은 반영되지 않는다.
향후 연구에서는 CLIP 기반 임베딩을 활용하거나 AI구글 이미지 검색과 같은 AI 시스템을 도입하여 사용자의 의미적 유사도 개선, 스타일, 계절 등의 속성 정보를 결합한 추천 모델, 사용자 선호도를 반영한 개인화 추천 시스템 등으로 확장하여 보다 정교한 추천 시스템을 구현하고자 한다.

부 록

자료 출처 -

< Python Code >

```
#그림2.1 카테고리별 데이터 수집
import os
import re
import time
import requests
BASE_URL = "https://filluminate.com"
HEADERS = {"User-Agent": "Mozilla/5.0 (compatible;
FashionProjectBot/1.0)"}]
CATEGORY_URLS = {
"top": [ # 상의
"https://filluminate.com/category/t-shirt/894/",
"https://filluminate.com/category/shirt/895/",
"https://filluminate.com/category/mtm-hd/897/",
"https://filluminate.com/category/knit/896/"],
"bottom": [ # 하의
"https://filluminate.com/category/bottom/893/",
"https://filluminate.com/category/jeans/986/",
"https://filluminate.com/category/pants/987/",
"https://filluminate.com/category/shorts/988/"],
"outerwear": [ # 겉옷
"https://filluminate.com/category/outer/614/"],
"hat": [ # 모자
"https://filluminate.com/category/cap/994/"],
"accessory": [ # 액세서리
"https://filluminate.com/category/acc/798/",
"https://filluminate.com/category/bag/996/"]
}def ensure_dir(path):os.makedirs(path, exist_ok=True)
def fetch_html(url):
try:resp = requests.get(url, headers=HEADERS, timeout=10)
resp.raise_for_status()return resp.text
except Exception as e:
print(f"⚠ 연결 실패: {url} / 에러: {e}")
return None
def extract_image_urls(html):
pattern = r"//filluminate\.com/web/product/extra/big/[^\s\"']+\.
matches = re.findall(pattern, html)
unique_urls = []
for m in matches:
full_url = "https:" + m if m.startswith("//") else m
if full_url not in unique_urls:
unique_urls.append(full_url)
return unique_urls
def download_image(img_url, save_dir, index):
ext = ".jpg"
filename = f"img_{index:04d}{ext}"
path = os.path.join(save_dir, filename)
if os.path.exists(path):
return
try:resp = requests.get(img_url, headers=HEADERS,
timeout=10)
if resp.status_code == 200:
with open(path, "wb") as f:
f.write(resp.content)
except Exception as e:
def main():root_dir = "data"
ensure_dir(root_dir)
for label, urls in CATEGORY_URLS.items():
save_dir = os.path.join(root_dir, label)
ensure_dir(save_dir)
img_count = 0
for base_url in urls:
for page in range(1, 4):
page_url = f"{base_url}?page={page}"
print(f"    Reading: {page_url}")
html = fetch_html(page_url)
if not html: continue
img_urls = extract_image_urls(html)
if not img_urls:
break
for img_url in img_urls:
img_count += 1
download_image(img_url, save_dir, img_count)
time.sleep(0.2)
if __name__ == "__main__":
main()
```

표2.1 카테고리별 파일 생성

```
import os
root_dir = "./data"
if not os.path.exists(root_dir):
total_count = 0
for category in os.listdir(root_dir):
category_path = os.path.join(root_dir, category)
if os.path.isdir(category_path):
count = len(os.listdir(category_path))
print(f"  🗁 {category}: {count}장")
total_count += count
print(f"\n🎉 총 {total_count}장의 옷 사진을 모았습니다!")
if total_count > 0:
```

표2.2 학습 / 테스트용 분리

```
import os
import shutil
import random
source_dir = "./data"
target_dir = "./dataset_final"
split_ratio = 0.8
if os.path.exists(target_dir):
shutil.rmtree(target_dir)
categories = [d for d in os.listdir(source_dir)
if os.path.isdir(os.path.join(source_dir, d))]
print(f"발견된 카테고리: {categories}")
total_train = 0
total_test = 0
for category in categories:
src_path = os.path.join(source_dir, category)
images = [f for f in os.listdir(src_path)
if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
random.shuffle(images)
train_count = int(len(images) * split_ratio)
train_images = images[:train_count]
test_images = images[train_count:]
for split_name, split_imgs in
[("train", train_images), ("test", test_images)]:
save_path = os.path.join(target_dir, split_name, category)
os.makedirs(save_path, exist_ok=True)
for img in split_imgs:
shutil.copy2(os.path.join(src_path, img),
os.path.join(save_path, img))
total_train += len(train_images)
total_test += len(test_images)
```

```
#그림 3.1 Epoch = 5
#그림 3.2
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader
import os
BATCH_SIZE = 32
LEARNING_RATE = 0.001
EPOCHS = 5
DEVICE = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
print(f"학습 장치: {DEVICE}")
transform = transforms.Compose([
transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])])
train_data = datasets.ImageFolder('./dataset_final/train',
transform=transform)
train_loader = DataLoader(train_data,
batch_size=BATCH_SIZE, shuffle=True)
class_names = train_data.classes
print(f"분류할 클래스: {class_names}")
model = models.resnet18(pretrained=True)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, len(class_names))
model = model.to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),
lr=LEARNING_RATE)
model.train()
for epoch in range(EPOCHS):
running_loss = 0.0
for inputs, labels in train_loader:
inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
print(f"[{epoch+1}/{EPOCHS}] Epoch 완료 | 평균 Loss:
{running_loss/len(train_loader):.4f}")
```

```
#그림3.3 Epoch = 10
#그림3.4
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader
import os
EPOCHS = 10
BATCH_SIZE = 32
LEARNING_RATE = 0.001
SAVE_NAME = "fashion_model_10epochs.pth"
DEVICE = torch.device
("cuda" if torch.cuda.is_available() else "cpu")
train_transform = transforms.Compose([
transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize
([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
train_data = datasets.ImageFolder
('./dataset_final/train', transform=train_transform)
train_loader = DataLoader
(train_data, batch_size=BATCH_SIZE, shuffle=True)
class_names = train_data.classes
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, len(class_names))
model = model.to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam
(model.parameters(), lr=LEARNING_RATE)
for epoch in range(EPOCHS):
model.train()
running_loss = 0.0
correct = 0
total = 0
for inputs, labels in train_loader:
inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()
epoch_acc = 100 * correct / total
print(f"[{epoch+1}/{EPOCHS}]
Loss: {running_loss/len(train_loader):.4f} |
정확도: {epoch_acc:.2f}%")
torch.save(model.state_dict(), SAVE_NAME)
```

```
#그림 3.5 Epoch = 20
#그림 3.6
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader
import os
BATCH_SIZE = 32
LEARNING_RATE = 0.001
EPOCHS = 20
DEVICE = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
train_transform = transforms.Compose([
transforms.Resize((224, 224)),
transforms.RandomHorizontalFlip(),
transforms.RandomRotation(15),
transforms.ColorJitter(brightness=0.2, contrast=0.2),
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])])
test_transform = transforms.Compose([
transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])])
train_data = datasets.ImageFolder('./dataset_final/train',
transform=train_transform)
test_data = datasets.ImageFolder('./dataset_final/test',
transform=test_transform)
train_loader = DataLoader(train_data,
batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data,
batch_size=BATCH_SIZE, shuffle=False)
class_names = train_data.classes
model = models.resnet18(pretrained=True)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, len(class_names))
model = model.to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),
lr=LEARNING_RATE)
for epoch in range(EPOCHS):
model.train()
running_loss = 0.0
correct = 0
total = 0
for inputs, labels in train_loader:
inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
```

```
optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()
epoch_loss = running_loss / len(train_loader)
epoch_acc = 100 * correct / total
print(f"[{epoch+1}/{EPOCHS}] Loss: {epoch_loss:.4f} |
정확도: {epoch_acc:.2f}%")
```

```
#그림3.7
#그림3.8
#그림3.9
#그림3.10
import torch
import torch.nn as nn
from torchvision import models, transforms, datasets
from torch.utils.data import DataLoader
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import os
import requests
from io import BytesIO
DEVICE = torch.device
("cuda" if torch.cuda.is_available() else "cpu")
DATA_DIR = './dataset_final/train'
MODEL_PATH = "fashion_model_v2.pth"
model = models.resnet18(pretrained=False)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 5)
try:model.load_state_dict
(torch.load(MODEL_PATH, map_location=DEVICE))
model.fc = nn.Identity()
model = model.to(DEVICE)
model.eval()
except FileNotFoundError:
print(f"❌ '{MODEL_PATH}' 파일이 없습니다.
학습 코드를 먼저 돌려주세요.")
exit()
```

```
transform = transforms.Compose([
transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])])
db_dataset = datasets.ImageFolder(DATA_DIR,
transform=transform)
db_loader = DataLoader(db_dataset, batch_size=32,
shuffle=False)
db_features = []
db_image_paths = [s[0] for s in db_dataset.samples]
with torch.no_grad():
for inputs, _ in db_loader:
outputs = model(inputs.to(DEVICE))
db_features.append(outputs.cpu().numpy())
db_features = np.concatenate(db_features, axis=0)
while True:print("\n" + "="*50)
url = input("👉 옷 이미지 링크를 붙여넣으세요 (그만하려면
'q' 입력): ").strip()
if url.lower() == 'q':
print("프로그램을 종료합니다.")
break
if not url: continue
try:response = requests.get(url, timeout=5)
response.raise_for_status()
query_image =
Image.open(BytesIO(response.content)).convert('RGB')
query_tensor =
transform(query_image).unsqueeze(0).to(DEVICE)
with torch.no_grad():
query_feature = model(query_tensor).cpu().numpy()
similarities = cosine_similarity(query_feature,
db_features)[0]
top_k_indices = similarities.argsort()[::-1][:5] # 상위 5개
plt.figure(figsize=(15, 6))
plt.subplot(1, 6, 1)
plt.imshow(query_image)
plt.title("Input Link")
plt.axis('off')
for i, idx in enumerate(top_k_indices):
path = db_image_paths[idx]
category = os.path.basename(os.path.dirname(path))
score = similarities[idx] * 100
print(f"  {i+1}위: {category} (유사도 {score:.1f}%)")
img = Image.open(path).convert('RGB')
plt.subplot(1, 6, i + 2)
plt.imshow(img)
plt.title(f"{i+1}st\n[{category}]")
plt.axis('off')
plt.tight_layout()
plt.show()
```